

Data Structures & Algorithms: Trees and Recursive Structures

Section 1: Binary Search Trees and Properties

A Binary Search Tree (BST) is a hierarchical node-based data structure where each node contains a key. The left subtree of a node contains only keys lesser than the node's key, and the right subtree contains only keys greater than the node's key. Both left and right subtrees must also be binary search trees.

In-order traversal of a BST produces elements in strictly ascending sorted order. The time complexity for searching, insertion, and deletion in a balanced BST is $O(\log n)$, whereas a degenerate or skewed tree degrades to $O(n)$ worst-case time complexity.

Section 2: AVL Trees and Rebalancing Rotations

An AVL tree is a strictly self-balancing binary search tree. For every node in an AVL tree, the heights of its two child subtrees differ by at most one (balance factor of -1, 0, or +1).

When an insertion or deletion causes the balance factor to become +2 or -2, rebalancing is performed using tree rotations:

1. Left Rotation (LL Case): Applied when a node is inserted into the right subtree of a right child.
2. Right Rotation (RR Case): Applied when a node is inserted into the left subtree of a left child.
3. Left-Right Rotation (LR Case): A double rotation where the left child undergoes a left rotation, followed by a right rotation at the root.
4. Right-Left Rotation (RL Case): A right rotation on the right child followed by a left rotation at the root.

Section 3: Recursion and Call Stack Dynamics

Recursion is a computational method where a function solves a problem by calling smaller instances of itself. Every valid recursive algorithm must define two components:

1. Base Case: The terminating condition that produces an answer directly without further recursive calls.
2. Recursive Step: The progression where the problem size is strictly decreased towards the base case.

When a recursive function executes, each active call allocates a stack frame on the call stack containing local variables and return addresses. Excessive depth without reaching the base case triggers a stack overflow exception.

Tail recursion occurs when the recursive call is the final action executed by the function, allowing compiler optimizations to reuse stack frames.